

## **TECNOLOGÍAS QUE INVESTIGUE QUE PODRIA UTILIZAR**

Para realizar el proyecto primero investigué un poco sobre las tecnologías que podrían ayudarme a cumplir con las funciones que se necesitan. No quería elegir las solamente porque ya las había utilizado antes, sino revisar qué opciones existen y cuáles se adaptan mejor a lo que se quiere hacer.

### **Python**

Para el proyecto decidí utilizar python porque considero que se adapta bien a las funciones que necesito realizar. Investigando encontré varias librerías que me pueden facilitar el trabajo, por ejemplo, psutil para obtener información del CPU, RAM, disco y red, smtplib para enviar las alertas por correo, y asyncio para manejar tareas que necesitan ejecutarse mientras el sistema sigue funcionando.

También puedo utilizar herramientas de python para conectarme con PostgreSQL y generar los reportes. Por eso considero que es una buena opción.

### **PostgreSQL**

Para la base de datos podría utilizar PostgreSQL.

El proyecto va a manejar bastante información y además va a haber diferentes datos relacionados entre sí, por ejemplo, las sucursales, equipos, técnicos, mantenimientos, servidores, alertas y registros del monitoreo.

Por eso considero que una base de datos relacional es una buena opción, ya que me permite organizar toda esta información y relacionarla correctamente. También tomé en cuenta que el sistema estará registrando información constantemente, principalmente las métricas de los servidores, por lo que la base de datos debe poder manejar una cantidad considerable de registros sin que las consultas se vuelvan demasiado pesadas.

La base de datos será creada y organizada por mí de acuerdo con la información que me proporcione la empresa y con las necesidades que se vayan definiendo durante el proyecto.

### **FastAPI**

Para la backend vi que puedo utilizar FastAPI, principalmente porque funciona con python y porque permite crear la comunicación entre las diferentes partes del sistema.

Además, investigando encontré que permite utilizar WebSockets, que son importantes para este proyecto porque el dashboard necesita recibir información del monitoreo en tiempo real. También puede ayudarme a mantener organizada la comunicación entre el sistema de monitoreo, la base de datos y la parte visual del proyecto.

## **Vue.js**

Para el dashboard investigue sobre Vue.js. También revisé otras opciones como Angular y React, pero considero que Vue puede ser una opción más sencilla para lo que necesito realizar. El dashboard tendrá que mostrar información que cambia constantemente, como el estado del CPU, RAM, disco y conexión de los servidores. Vue permite actualizar la información de la pantalla de forma dinámica, por lo que considero que se adapta bien a esta parte del proyecto.

## **WebSockets**

Para actualizar la información del dashboard en tiempo real estoy considerando utilizar WebSockets. Primero investigué también la opción de utilizar polling, que básicamente consiste en que el dashboard pregunte cada cierto tiempo si existen nuevos datos. Sin embargo, para este proyecto me parece mejor utilizar WebSockets porque permite mantener una conexión y recibir información cuando hay nuevos datos. Esto puede ser útil para que el estado de los servidores se pueda mostrar de una manera más inmediata.

## **psutil**

Para obtener la información del servidor utilizaría psutil, que es una biblioteca de Python. La elegí porque permite consultar diferentes recursos del equipo, como:

- CPU
- RAM
- Disco
- Red
- Procesos

Esto se relaciona directamente con lo que se necesita monitorear en el proyecto, por lo que considero que es una herramienta bastante adecuada.

## **SMTP y correo electrónico**

Para las alertas por correo utilizaría SMTP, junto con las herramientas que proporciona Python para enviar correos. Esto sería necesario porque el sistema debe poder avisar cuando detecte algún problema, por ejemplo, si un respaldo falla o si algún recurso del servidor llega a un nivel determinado. También considero importante que los datos del correo y los límites de las alertas no estén escritos directamente en el código. Por eso utilizaría un archivo .env para guardar este tipo de configuración y poder modificarla más fácilmente.

## Simulación del servidor

Investigué algunas opciones para simular el servidor, como VirtualBox, Docker, Hyper-V y también hacerlo mediante un servidor local. Esta última opción ya la había visto en otros proyectos de la universidad, donde levantábamos un servidor desde la terminal utilizando Node.js.

### TECNOLOGÍAS PROPUESTAS

| Área                | Tecnología                          | Función                                                     |
|---------------------|-------------------------------------|-------------------------------------------------------------|
| Lenguaje            | <b>Python</b>                       | Desarrollo principal del sistema y monitoreo                |
| Monitoreo           | <b>psutil</b>                       | Obtener CPU, RAM, disco y red                               |
| Base de datos       | <b>PostgreSQL</b>                   | Almacenar mantenimiento, métricas, respaldos, alertas, etc. |
| Backend/API         | <b>FastAPI</b>                      | Comunicación entre BD, monitoreo y frontend                 |
| Dashboard           | <b>Vue.js</b>                       | Interfaz web                                                |
| Tiempo real         | <b>WebSockets</b>                   | Actualizar métricas sin estar consultando constantemente    |
| Correo              | <b>SMTP + smtplib</b>               | Enviar alertas                                              |
| Configuración       | <b>.env / variables de entorno</b>  | SMTP, límites de alerta y configuración                     |
| Tareas secundarias  | <b>FastAPI BackgroundTasks</b>      | Generar reportes sin congelar la interfaz                   |
| Ejecutable          | <b>PylInstaller</b>                 | Empaquetar componentes Python                               |
| Servidor de pruebas | <b>VirtualBox + máquina virtual</b> | Simular un servidor                                         |

## **REQUERIMIENTOS FUNCIONALES**

### **1. Gestión de mantenimiento**

El sistema deberá permitir registrar:

- fecha;
- equipo;
- problema;
- solución aplicada;
- tiempo empleado;
- técnico asignado.

También deberá permitir consultar los registros existentes.

### **2. Gestión de equipos**

El sistema deberá permitir registrar y consultar información de los equipos tecnológicos de las diferentes sucursales.

### **3. Monitoreo de servidores**

El sistema deberá obtener información de:

- CPU;
- memoria RAM;
- almacenamiento;
- conexión de red;
- disponibilidad/estado general del servidor.

### **4. Dashboard**

El sistema deberá mostrar el estado de los servidores mediante un dashboard centralizado.

El estado podrá representarse mediante:

- **Correcto**
- **Advertencia**
- **Problema**
- **Desconectado**

### **5. Actualización en tiempo real**

El dashboard deberá recibir actualizaciones de las métricas mediante **WebSockets**, evitando depender exclusivamente de consultas periódicas.

## **6. Detección de problemas**

El sistema deberá identificar cuando una métrica supere los límites configurados.

CPU 70% -Correcto

CPU 70–90% -Advertencia

CPU > 90%-Problema

## **7. Alertas**

Cuando se detecte una condición que requiera atención, el sistema deberá registrar el evento y, cuando corresponda, enviar una notificación mediante correo electrónico.

## **8. Control de respaldos**

El sistema deberá registrar:

- fecha;
- hora;
- servidor;
- tamaño;
- resultado del respaldo.

Si un respaldo falla, deberá generar una alerta por correo.

## **9. Reportes**

El sistema deberá permitir generar información estadística relacionada con:

- equipos con mayor cantidad de fallas
- técnicos con mayor cantidad de servicios
- tiempo promedio de reparación
- incidentes
- estado de servidores;
- respaldos

Los reportes podrán generarse en:

- **PDF** presentación formal.
- **CSV** análisis/exportación de datos.

## **10. Historial**

El sistema deberá conservar el historial de las métricas, mantenimientos, respaldos y eventos para permitir consultas posteriores

## **REQUERIMIENTOS NO FUNCIONALES**

### **1. Disponibilidad**

El sistema de monitoreo deberá poder ejecutarse de manera continua.

### **2. Tolerancia a fallos de red**

### **3. Configuración externa**

Los parámetros sensibles y configurables no deberán estar escritos directamente en el código

Ejemplo:

```
SMTP_HOST  
SMTP_PORT  
SMTP_USER  
ALERT_CPU  
ALERT_RAM  
ALERT_DISK
```

### **4. Rendimiento**

El monitoreo deberá utilizar los recursos del servidor de manera eficiente y evitar generar una carga innecesaria.

### **5. Base de datos**

La base de datos deberá estar diseñada para soportar grandes cantidades de registros históricos de monitoreo.

### **6. Reportes**

La generación de reportes deberá realizarse en segundo plano para evitar que la interfaz quede bloqueada mientras se procesa información.

### **7. Portabilidad**

El componente de monitoreo desarrollado en python deberá poder empaquetarse como ejecutable mediante PyInstaller, facilitando su instalación en el equipo donde se utilizará.